

Exercício de Teste de API - Carros

Objetivo: Este exercício tem como objetivo familiarizar os participantes com testes de API RESTful usando a API de Carros. Você irá realizar testes de Listagem, Consulta, Criação, Atualização e Exclusão (CRUD) de carros, validando códigos de status HTTP e o conteúdo das respostas.

Endpoint Base: <http://localhost:3001/>

Endpoints a serem utilizados:

- Listar todos os carros: GET /carros
 - Obter detalhes de um carro: GET /carros/{id}
 - Criar um novo carro: POST /carros
 - Atualizar um carro existente: PUT /carros/{id}
 - Excluir um carro: DELETE /carros/{id}
-

Instruções

1. Teste de Listagem (GET /carros)

Objetivo: Obter a lista de todos os carros cadastrados.

Passos:

1. Execute uma solicitação GET para /carros.
 2. Verifique se a resposta tem o código de status **200 OK**.
 3. Analise o formato da resposta para garantir que todos os campos esperados (id, marca, modelo, ano, cor, preco) estejam presentes.
-

2. Teste de Consulta (GET /carros/{id})

Objetivo: Obter detalhes de um carro específico.

Passos:

1. Escolha um id de um carro existente.
2. Execute uma solicitação GET para /carros/{id} (substitua {id} pelo ID escolhido).
3. Verifique se a resposta tem o código de status **200 OK**.
4. Confirme se os detalhes do carro correspondem aos dados esperados.

3. Teste de Criação (POST /carros)

Objetivo: Criar um novo carro no sistema.

Passos:

1. Execute uma solicitação POST para /carros.
2. No corpo da requisição, envie os dados do carro em formato JSON:

```
{  
  "marca": "Toyota",  
  "modelo": "Corolla",  
  "ano": 2023,  
  "cor": "Prata",  
  "preco": 25000  
}
```

3. Verifique se a resposta tem o código de status **201 Created**.
4. Confirme se a resposta contém o carro criado com um id atribuído.

4. Teste de Atualização (PUT /carros/{id})

Objetivo: Atualizar as informações de um carro existente.

Passos:

1. Escolha um id de um carro existente.
2. Execute uma solicitação PUT para /carros/{id}.
3. No corpo da requisição, envie os campos que deseja atualizar:

```
{  
  "marca": "Toyota",  
  "modelo": "Corolla XEI",  
  "ano": 2023,  
  "cor": "Preto",  
  "preco": 27000  
}
```

}

4. Verifique se a resposta tem o código de status **200 OK**.
5. Confirme se os dados do carro foram atualizados corretamente.

5. Teste de Exclusão (DELETE /carros/{id})

Objetivo: Excluir um carro do sistema.

Passos:

1. Escolha um id de um carro existente.
2. Execute uma solicitação DELETE para /carros/{id}.
3. Verifique se a resposta tem o código de status **200 OK** e contém a mensagem de confirmação.
4. Tente consultar o carro excluído usando GET /carros/{id} e verifique se a resposta é um código de status **404 Not Found**.

Validações Adicionais

6. Teste de Erro ao Consultar Carro Inexistente

Objetivo: Verificar o comportamento da API ao solicitar um carro que não existe.

Passos:

1. Escolha um id que não existe no sistema (por exemplo, 9999).
2. Execute uma solicitação GET para /carros/{id_inexistente}.
3. Verifique se a resposta tem o código de status **404 Not Found**.
4. Confirme se a resposta contém uma mensagem de erro apropriada, como "Carro não encontrado".

7. Teste de Erro ao Criar Carro com Dados Incompletos

Objetivo: Testar a validação de dados na criação de um novo carro.

Passos:

1. Execute uma solicitação POST para /carros.

2. No corpo da requisição, envie dados incompletos (por exemplo, sem o campo marca):

```
{  
  "modelo": "Civic",  
  "ano": 2022,  
  "cor": "Azul",  
  "preco": 23000  
}
```

3. Observe o comportamento da API (a implementação atual não possui validação completa, mas este teste é importante para verificar robustez).
4. Idealmente, a resposta deveria ter o código de status **400 Bad Request** e uma mensagem indicando quais campos estão faltando.

8. Teste de Valores Inválidos

Objetivo: Verificar como a API lida com dados inválidos.

Passos:

1. Execute uma solicitação POST para /carros com valores inválidos:

```
{  
  "marca": "Honda",  
  "modelo": "Fit",  
  "ano": "dois mil e vinte",  
  "cor": "Verde",  
  "preco": "caro"  
}
```

2. Verifique o comportamento da API ao receber tipos de dados incorretos.
3. Analise se há tratamento de erros adequado para valores não numéricos nos campos ano e preco.

Dicas para os testes

- Use o **Postman** para executar as requisições.
- Documente os resultados de cada teste, incluindo código de status e corpo da resposta.
- Teste cenários positivos (dados válidos) e negativos (dados inválidos ou ausentes).
- Verifique a documentação Swagger em <http://localhost:3001/api-docs> para referência dos endpoints.